

EXPERIMENT 3 : 8-PUZZLE PROBLEM

Aim

To develop a Python program that solves the 8-Puzzle Problem using the Breadth-First Search (BFS) algorithm to find the shortest path from the initial state to the goal state.

Objective

- To understand the 8-Puzzle problem in Artificial Intelligence.
- To apply the Breadth-First Search (BFS) algorithm.
- To find the shortest sequence of moves from the initial state to the goal state.
- To display each intermediate state until the goal is reached.

Algorithm

Step 1: Start.

Step 2: Define the initial state and goal state of the puzzle.

Step 3: Create a queue and insert the initial state.

Step 4: Mark the initial state as visited.

Step 5: Remove the first state from the queue.

Step 6: Check whether it is the goal state.

- If **Yes**, display the solution and stop.
- If **No**, continue.

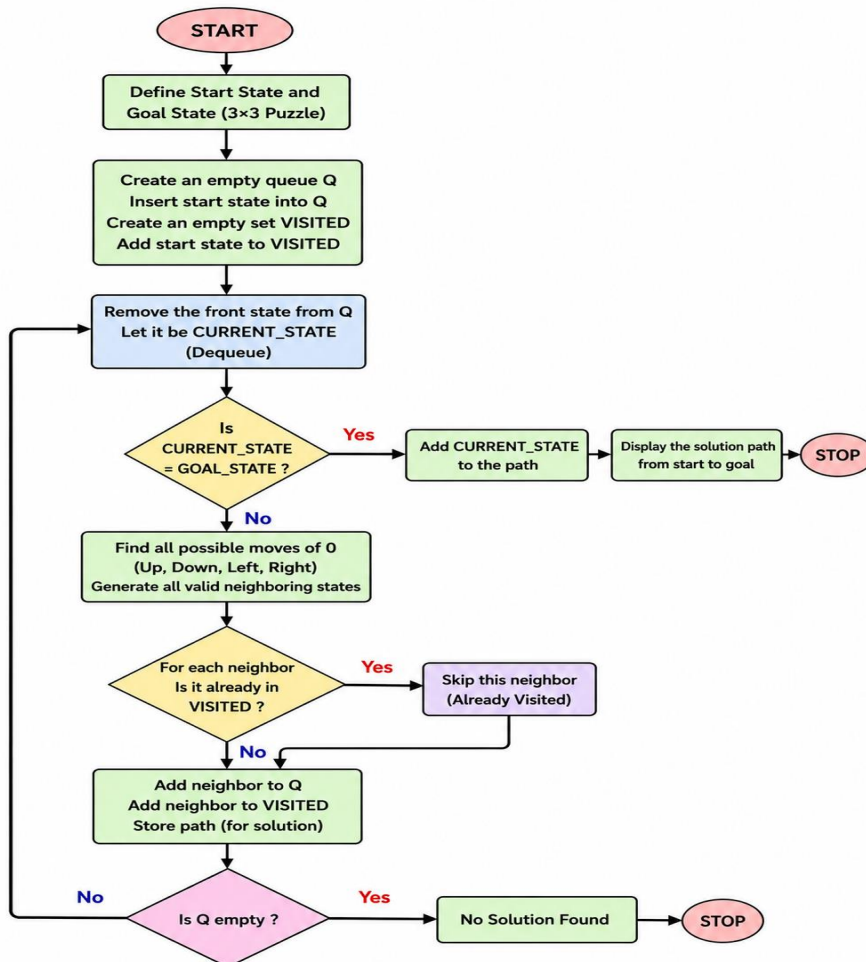
Step 7: Find all valid neighboring states by moving the blank tile (0).

Step 8: Add all unvisited neighboring states to the queue.

Step 9: Repeat Steps 5–8 until the goal state is found or the queue becomes empty.

Step 10: Stop.

Flow chart :



Source code:

```
from collections import deque
```

```
goal = [[1, 2, 3],  
        [4, 5, 6],  
        [7, 8, 0]]
```

```
start = [[1, 2, 3],  
        [4, 0, 6],  
        [7, 5, 8]]
```

```

def board_to_tuple(board):
    return tuple(tuple(row) for row in board)

def find_blank(board):
    for i in range(3):
        for j in range(3):
            if board[i][j] == 0:
                return i, j

def get_neighbors(board):
    x, y = find_blank(board)
    moves = [(-1,0), (1,0), (0,-1), (0,1)]
    neighbors = []

    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:

            new_board = [row[:] for row in board]
            new_board[x][y], new_board[nx][ny] = new_board[nx][ny],
new_board[x][y]
            neighbors.append(new_board)

    return neighbors

def bfs(start, goal):
    queue = deque([(start, [])])
    visited = set()

    while queue:
        board, path = queue.popleft()

        if board == goal:
            return path + [board]

        state = board_to_tuple(board)
        if state in visited:
            continue

```

```

        visited.add(state)

    for neighbor in get_neighbors(board):
        queue.append((neighbor, path + [board]))

    return None

solution = bfs(start, goal)

if solution:
    print("Solution Found:\n")
    for step in solution:

        for row in step:
            print(row)
        print()
else:
    print("No Solution Found")

```

output:

```
Solution Found:
```

```
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]
```

```
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]
```

```
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

```
Process finished with exit code 0
```

Result :

The Python program successfully solves the 8-Puzzle Problem using the Breadth-First Search (BFS) algorithm and displays the sequence of puzzle states from the initial state to the goal state.